



T.C. İSTANBUL KÜLTÜR ÜNİVERSİTESİ

COM5005 – WEB PROGRAMMING

LAB 08 – ASP.NET MVC5 VIEW & MODEL

After completing this Lab, you will be able to

- Razor View Engine
- Preparing Models
- Database Connection

PROCEDURE 1 – @Razor View Engine@

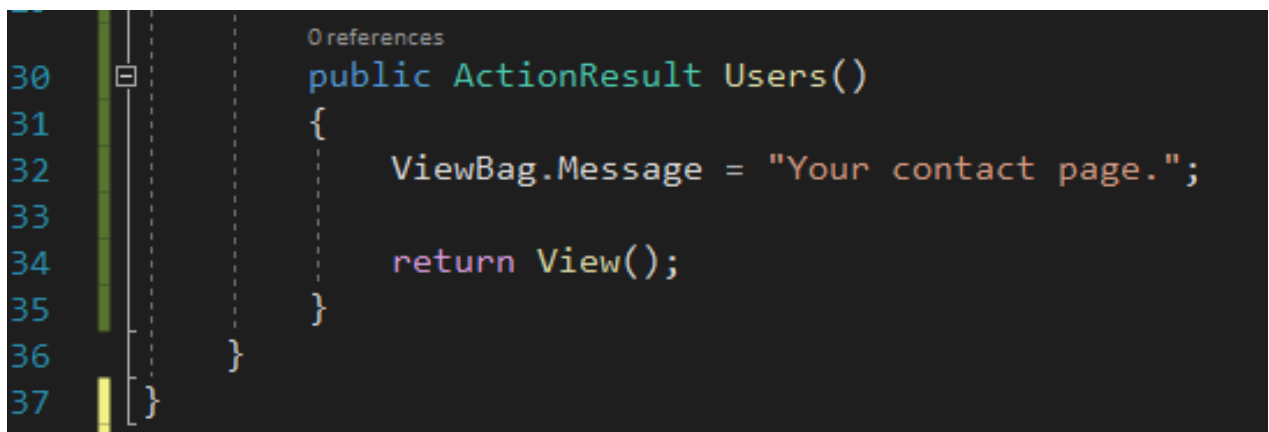
Razor View Engine; Views also allow the use of C# along with HTML, CSS, JavaScript. HTML, CSS and JavaScript codes written to Views work on the client side. C# codes run on the server side and the result is sent to the client.

- The code you write should always be in @{ ... }.
- Semicolon ";" at the end of your commands must exist.
- The extension of the files you use should be .cshtml.
- Your variables must be defined with the "var" keyword.
- Your inline expressions (variables and functions) must start with "@".
- Your C# codes are case sensitive.

Step 1 – Razor Code Block.

In order to write C# codes in View, we need to create a Razor code block with "@{ }". In our example, we defined 3 variables in the Razor code block. In order to access the variables that we defined in the Razor code block outside of the code block, we need to put an "@" sign in front of them.

Step 2 – Create new MVC project. And Create the "Users(): Action Result" in HomeController

A screenshot of a code editor with a dark background. On the left, line numbers 30 through 37 are visible. The code is written in C#. It shows a method named 'Users()' which returns an 'ActionResult'. Inside the method, 'ViewBag.Message' is set to 'Your contact page.' and then 'return View();' is called. There is a '0 references' label above the method signature. The code is as follows:

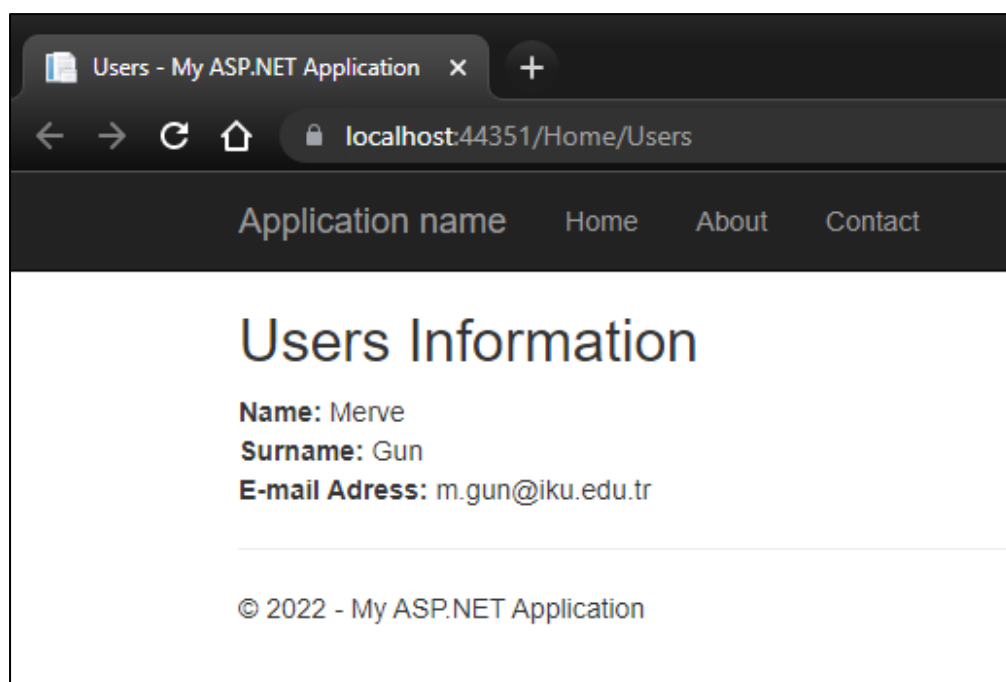
```
30 public ActionResult Users()  
31 {  
32     ViewBag.Message = "Your contact page.";  
33  
34     return View();  
35 }  
36  
37 }
```

Step 3 – Users.cshtml view was created automatically.

Let's write the following code in the page and run it.

```
1
2  @{
3      ViewBag.Title = "Users";
4  }
5
6  <h2>Users Information</h2>
7
8  @{
9      string name = "Merve";
10     string surName = "Gun";
11     string eMail = "m.gun@iku.edu.tr";
12 }
13 <b>Name:</b> @name
14 <br />
15 <b>Surname:</b> @surName
16 <br />
17 <b>E-mail Adress:</b> @eMail
18 <br />
19
```

** Execute the code and review the results.



Step 4 – Razor Code Line.

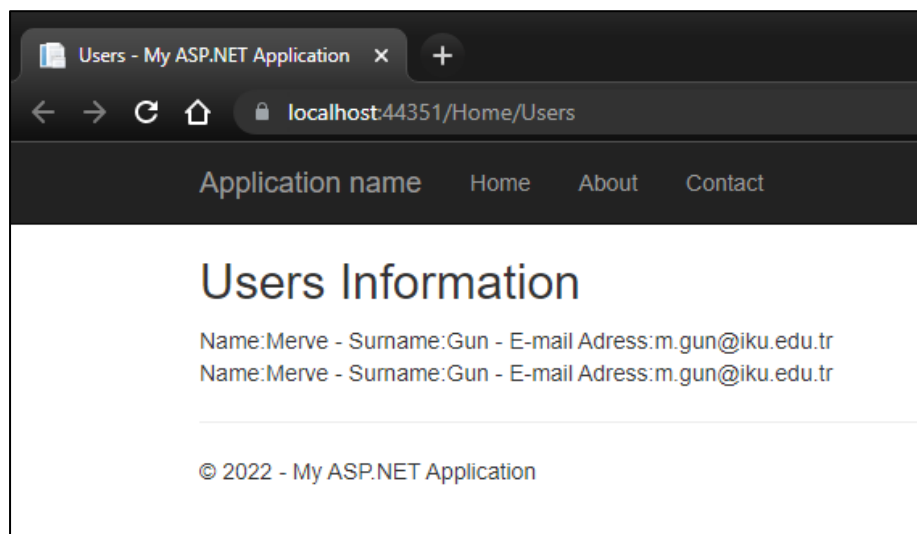
We can write one line of C# code in “@ ()”.

```

1
2 @{}
3     ViewBag.Title = "Users";
4 }
5
6 <h2>Users Information</h2>
7
8 @{}
9     string name1 = "Merve";
10    string surName1 = "Gun";
11    string eMail1 = "m.gun@iku.edu.tr";
12 }
13
14 @String.Format("Name:{0} - Surname:{1} - E-mail Adress:{2}", name1, surName1, eMail1))
15 <br />
16 @* Alternative Way of Use *@
17 @("Name:" + name1 + " - " + "Surname:" + surName1 + " - " + " E-mail Adress:" + eMail1)
18

```

** Execute the code and review the results.



Step 5 – Control Structures and Use of Loops.

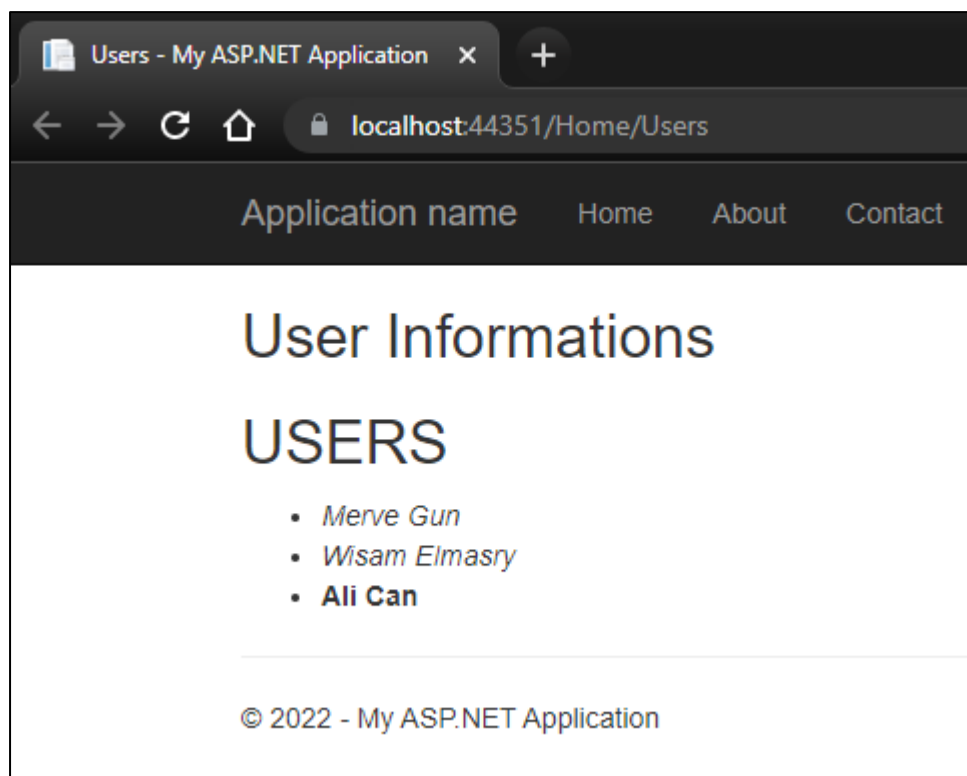
Thanks to Razor, we can use the loop and control structures of the C# language (For, Foreach, While, Switch-Case, If-Else...) anywhere on our page.

```

1
2  @{
3      ViewBag.Title = "Users";
4  }
5
6  <h2>User Informations</h2>
7
8  @{
9      string[] Users = new string[] { "Merve Gun", "Wisam Elmasry", "Ali Can" };
10 }
11 <h2>USERS</h2>
12 <ul>
13     @foreach (string user in Users)
14     {
15         if (user == "Ali Can")
16         {
17             <li><b>@user</b></li>
18         }
19         else
20         {
21             <li><i>@user</i></li>
22         }
23     }
24 </ul>
25

```

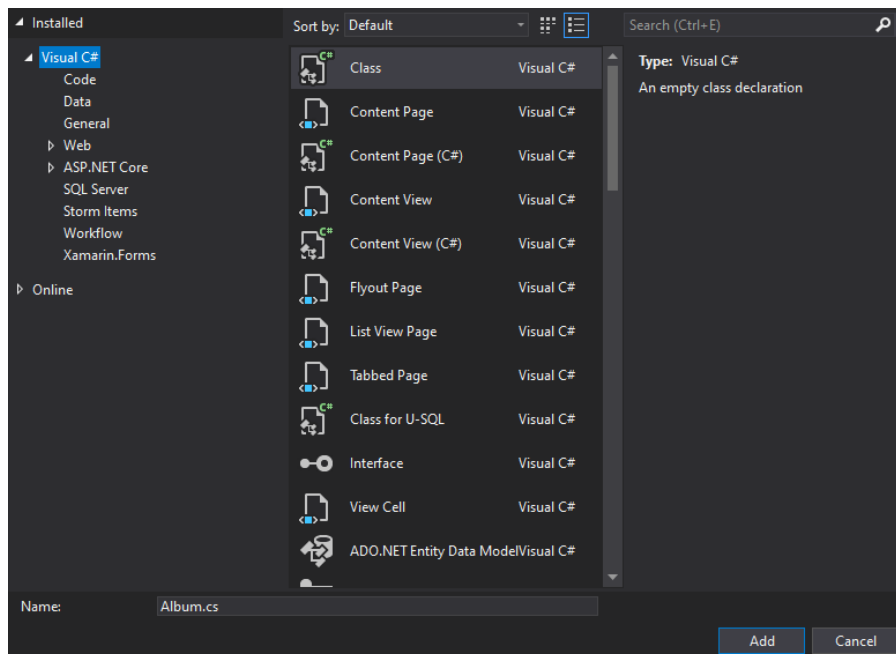
** Execute the code and review the results.



PROCEDURE 2 – Creating the MusicStore Project and Preparing Models.

We will create a music store website.

Step 1 - To add a new Album class to the Models folder, right-click the Models folder, select Add... Class, and name the **class Album**.

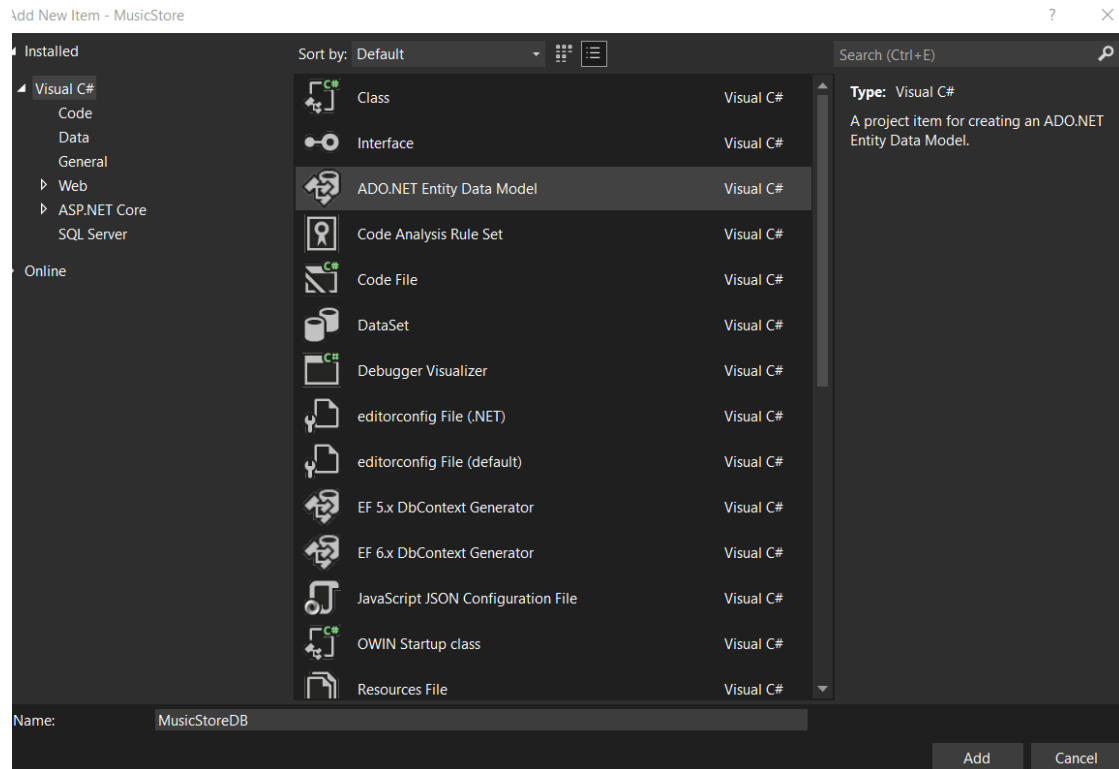


Step 2 - Let's add the parameters of the Album class.

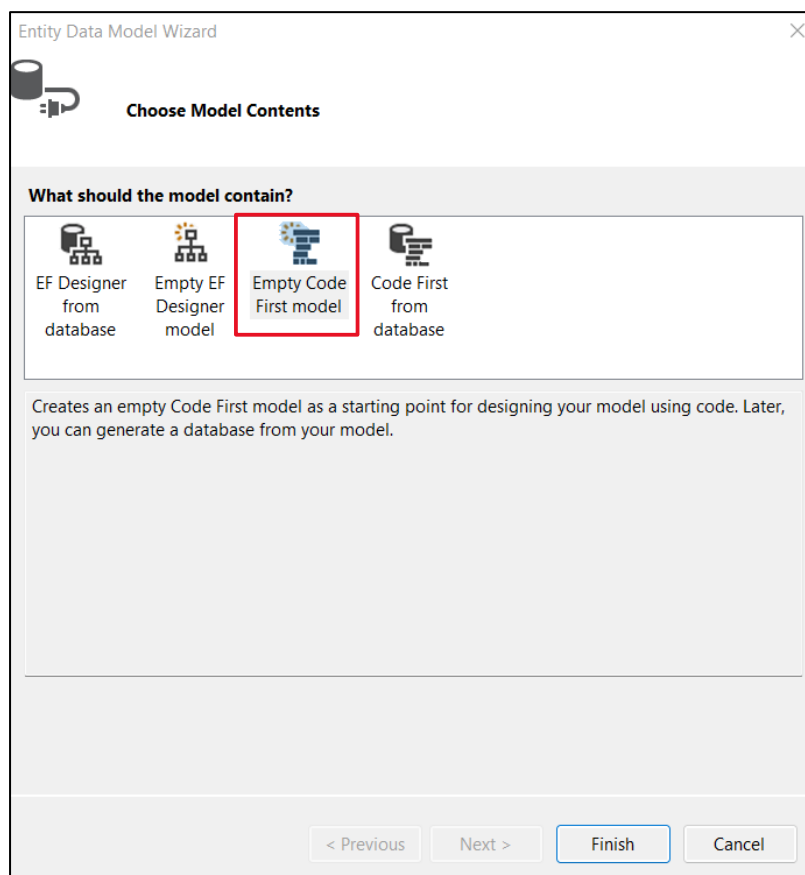
```
namespace MusicStore1.Models
{
    7 references
    public class Album
    {
        0 references
        public virtual int AlbumId { get; set; }
        0 references
        public virtual string Genre { get; set; }
        0 references
        public virtual string Artist { get; set; }
        0 references
        public virtual string Title { get; set; }
        0 references
        public virtual decimal Price { get; set; }
    }
}
```

Step 3 - We create ADO.NET Entity Data Model to establish database connection.

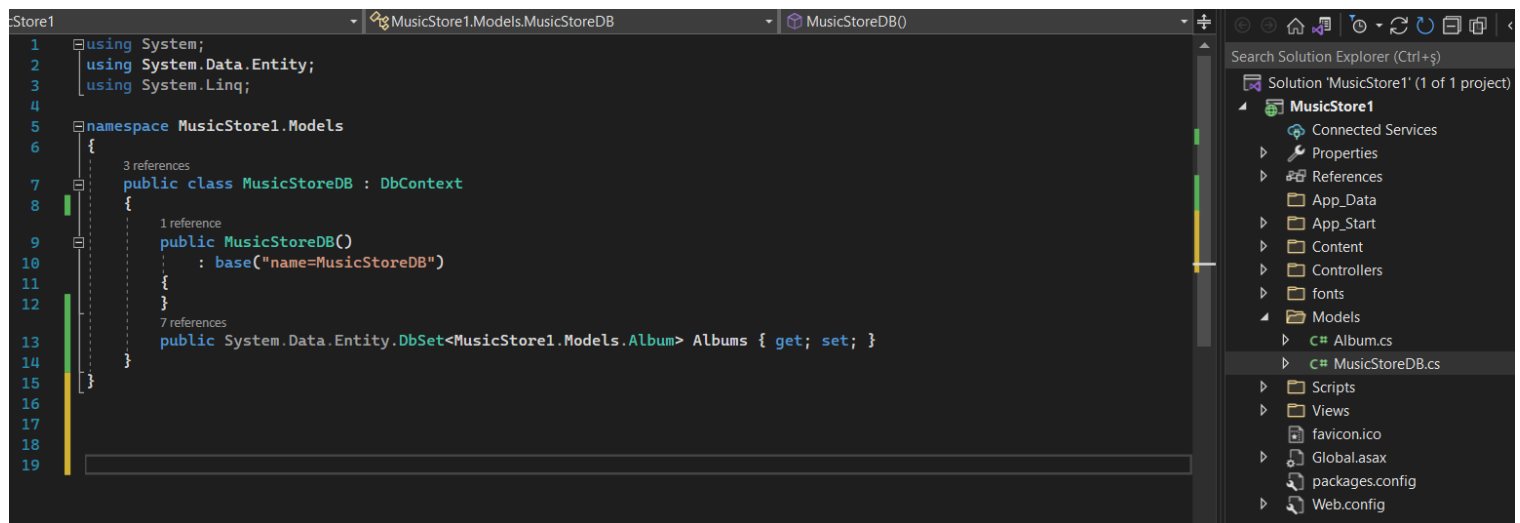
Right click Models >> Add >> Class >> ADO.NET Entity Data Model. We call it **MusicStoreDB**.



Step 4 - In the window that opens, we choose **Empty Code First Model** and Save.



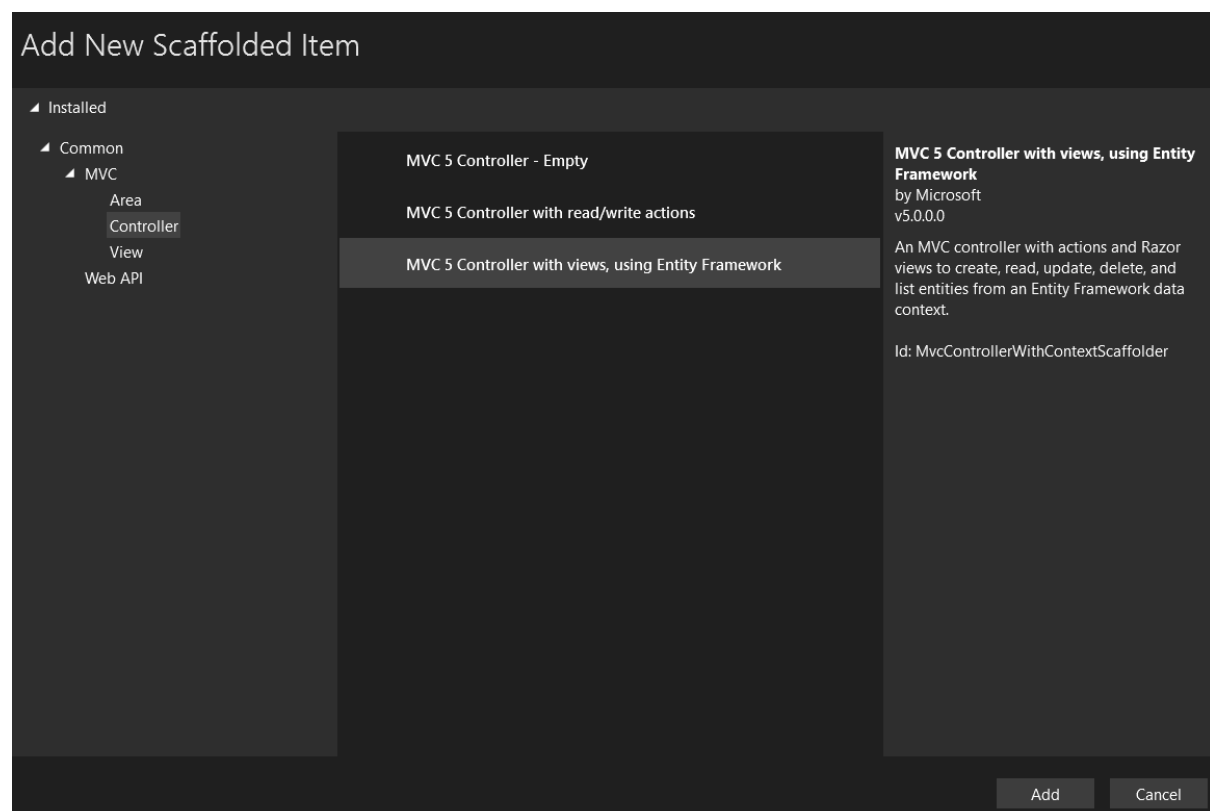
Step 5 - Through the "MusicStoreDB : DbContext" Class, we are connecting the model classes to the database.



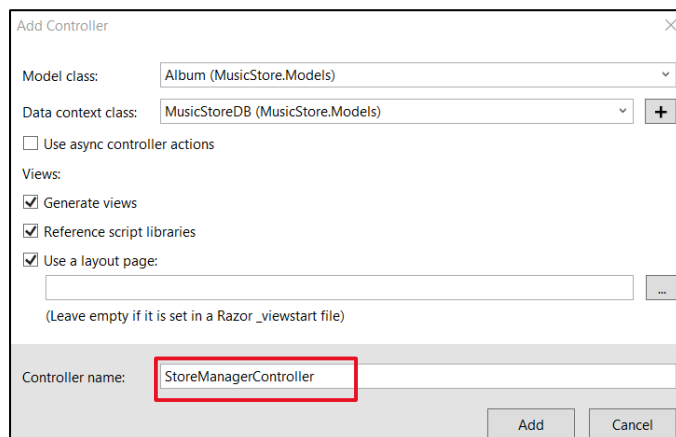
Step 6 - Store Manager Controller.

First running the project as it is and then returning to the code again;

Now we create new "**MVC5 Controller with views, using Entity Framework**" to access model classes.

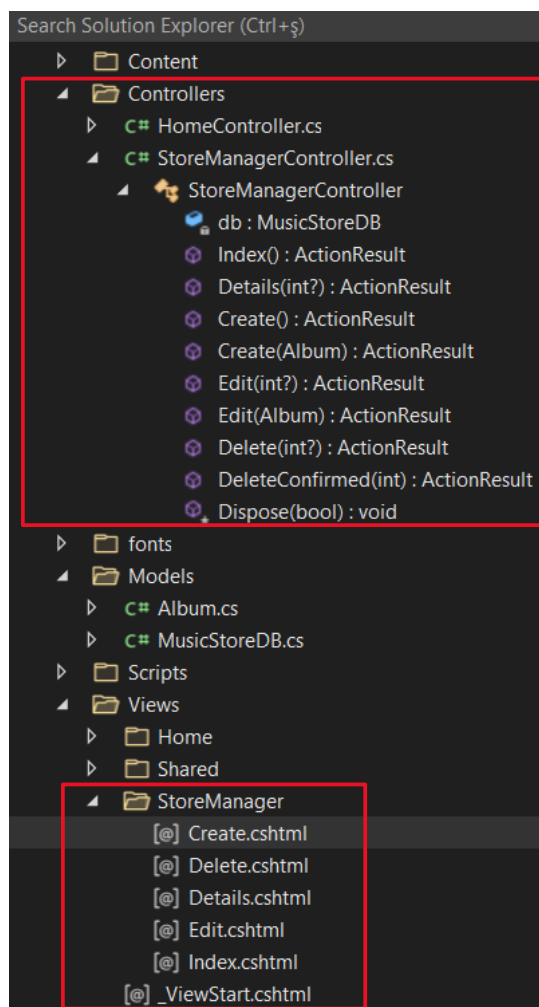


In the window that opens, we select the following information.



When we check the Solution Explorer, you will see that many **Action Functions** such as **Index, Create, Edit, Delete** are created automatically in the **StoreManager controller**.

In the same way, you will see that **the Views of each Action Result Function** are automatically created.



Step 7 - Run and inspect the Create.cshtml, Index.cshtml, Details.cshtml and Edit.cshtml view pages.

Create.cshtml

[Application name](#) [Home](#) [About](#) [Contact](#)

Create

Album

Genre	
Artist	
Title	
Price	

Create

[Back to List](#)

© 2022 - My ASP.NET Application

Index.cshtml

[Application name](#) [Home](#) [About](#) [Contact](#)

Index

[Create New](#)

Genre	Artist	Title	Price

© 2022 - My ASP.NET Application

Edit.cshtml

Application name Home About Contact

Edit

Album

Genre	Pop
Artist	Sezen Aksu
Title	Gülümse
Price	250.00

Save

[Back to List](#)

© 2022 - My ASP.NET Application

Details.cshtml

Details

Album

Genre	Pop
Artist	Sezen Aksu
Title	Gülümse
Price	250.00

[Edit](#) | [Back to List](#)

© 2022 - My ASP.NET Application

T.A. Merve GÜN